

Guía de Ejercicios 5

Tecnología Digital VI

Juan Ignacio Elosegui

Universidad Torcuato Di Tella

Junio 2026

Fundamentos de Redes Convolucionales (CNNs)

Ejercicio 1

Las redes fully-connected tienen la limitación que es que ignoran la estructura espacial: procesa los píxeles como si todos fuesen independientes entre sí, y este no suele ser el caso.

Una CNN explota la relación entre píxeles vecinos usando filtros (kernels) que aprenden los patrones de cada subconjunto cuadrado de píxeles.

Estos kernels se inicializan random y se ajustan via backpropagation. Este mismo kernel se aplica en todas las posiciones de la imagen, lo que reduce la cantidad de parámetros respecto a una fully-connected y además detecta los mismos patrones sin importar en qué posición se aplique. Antes, cada píxel tenía su propio peso hacia adelante (FC), ahora se comparten (con una CNN).

Ejercicio 2

Padding consiste en rellenar bordes con ceros para controlar el tamaño de salida.

Stride indica cuánto se mueve el filtro. Un *stride* más grande implica una salida más chica.

Usando la fórmula:

$$O = \frac{I - K + 2K}{S} + 1$$
$$\Rightarrow O = \frac{28 - 3 + 2 \times 1}{2} + 1$$
$$\therefore O = 14$$

Por lo tanto, la salida será de 14×14 .

Ejercicio 3

Ejercicio 4

La capa de *pooling* sirve para resumir información de un grupo de píxeles vecinos. Estas capas achican la dimensión (que ahorra espacio y cómputo), aumentan el campo receptivo y da robustez ante la traslación (un desplazamiento probablemente no cambie el resultado del pooling).

Arquitecturas famosas y Transfer Learning

Ejercicio 5

AlexNet aportó la posibilidad de entrenarse con múltiples GPUs, incluyó capas de normalización, data augmentation, dropout...

Ejercicio 6

La **ResNet** mitigó el problema de los vanishing gradients porque al agregar una skip connection cuyo gradiente es 1 (por ser la identidad), los gradientes no se pueden desvanecer.

Ejercicio 7

Transfer learning consiste en reutilizar capas preentrenadas en un dataset grande para otro problema distinto. Se puede optar por esta opción porque entrenar desde cero es muy costoso, por lo que reciclar las capas entrenadas y cambiar únicamente la última fully-connected acorde a lo que requiera el problema a resolver.

Se puede reentrenar la nueva capa y congelar todo lo otro hacer fine tuning de algunas capas previas.

Detección de objetos y segmentación (I2I)

Ejercicio 8

R-CNN usa regiones candidatas en las cuales se hace una inferencia. Se toma una imagen como input, se extraen regiones (≈ 2000) donde pueden haber objetos, y cada región pasa por una CNN que es sometida a un problema de clasificación. Se suele preferir una R-CNN antes que un algoritmo de fuerza bruta como uno de *sliding window*.

YOLO, por otro lado, es un algoritmo aún más eficiente. En una sola pasada, se pasa de los píxeles a las coordenadas de las *bounding boxes* con sus respectivas clases. Predice etiquetas de objetos y ubicaciones de las bounding boxes. Su función de pérdida penaliza errores de ubicación de las bounding boxes, tamaños de las bounding boxes, su tamaño y errores de clasificación.

Ejercicio 9

La imagen se divide en una grilla de $S \times S$ celdas. Cada una arroja B bounding boxes con cinco valores: x , y , w (ancho de la caja), *confidence* más C probabilidades de clase. El output formalizado es:

$$S \times S \times (B \times 5 + C)$$

El procedimiento de **Non-Max Suppression** hace que se tome la bounding box con mayor confianza para descartar cualquier otra con la cual compartan alta intersección sobre unión (IoU).

Ejercicio 10

Dice es una medida de concordancia entre las predicciones de la red y las anotaciones hechas a mano (la “ground truth”). Se prefiere Dice por sobre cross-entropy porque las clases suelen estar muy desbalanceadas (mucho fondo y poco objeto), entonces mira únicamente las áreas que la red acertó:

$$\text{Dice} = 2 \frac{|X \cap Y|}{|X| + |Y|}$$

Se puede referir a $|X|$ al área predicha y a $|Y|$ como el área real. El numerador $|X \cap Y|$ hace referencia al área como *True Positive*.

Interpretabilidad y manipulación

Ejercicio 11

Los **Saliency Maps** son cálculos de sensibilidad de predicción en backpropagation respecto a cada píxel. Los píxeles con mayor gradiente son los más importantes para predecir la clase. El input ahora es una variable.

Grad-CAM hace lo mismo pero no píxel a píxel, sino parche a parche. Calcula la importancia para cada uno generando un mapa de calor sobre la imagen.

Qué ventaja trae esto?

Ejercicio 12

Se busca el mínimo cambio en píxeles para que la red clasifique una clase como otra distinta. Se fija la nueva clase objetivo y se modifica la imagen de manera iterativa para maximizar esa probabilidad. Modificar un píxel puede no hacer la diferencia, pero modificar cien sí.

Ejercicio 13

Style Transfer consiste en transferir el estilo visual de una imagen a otra.

Primero, se optimiza una imagen intermedia I minimizando la función:

$$\mathcal{L}_{\text{total}} = \alpha \cdot \mathcal{L}_{\text{content}} + \beta \cdot \mathcal{L}_{\text{style}}$$